



STATE UNIVERSITY OF ZANZIBAR (SUZA)

School of Computing, Communication and Media Studies (SCCMS)

Department of Computer Science & Information Technology

Web Technologies

SOFTWARE DESIGN DESCRIPTION (SDD)

PROJECT TITLE: University Innovation Connect (UIC)

A Web-Based Collaborative Platform for Student Idea Sharing and Lecturer Feedback at SUZA

SN	Student Name	Registration number
1	Khadija Ali Said	24BCS026
2	Rahma Suleiman Abdalla	24BIT038

Academic Year: 2025/2026

Supervisor: Massoud Hamad Mmmaga

Submission Date: June 2026

Document Revision History

Version	Date	Author	Description of Change
0.1	29 June	Khadija Ali Said Rahma Suleiman Abdalla	Initial draft

TABLE OF CONTENT

Table of Contents

1. INTRODUCTION.....	6
1.1 Purpose.....	6
1.2 Scope.....	6
1.3 Definitions, Acronyms and Abbreviations.....	7
1.4 References.....	7
2. DESIGN STAKEHOLDERS AND THEIR CONCERNS.....	8
3. DESIGN OVERVIEW AND RATIONALE.....	9
3.1. Architectural Style.....	9
Presentation Layer (Frontend).....	9
The presentation layer is developed using React.js. It provides the graphical user interface (GUI) through which students, lecturers, and administrators interact with the system. The frontend communicates with the backend through RESTful APIs using HTTP requests and responses in JSON format.....	9
Application Layer (Backend).....	9
Data Layer (Database).....	9
The data layer uses PostgreSQL as the relational database management system. It stores all application data, including user accounts, innovation ideas, comments, votes, feedback, and notifications.....	9
Architecture Justification.....	10
System Architecture.....	10
.....	10
3.2 Design Principles and Patterns.....	10
Separation of Concerns (SoC).....	10
Single Responsibility Principle (SRP).....	11
Don't Repeat Yourself (DRY).....	11
Model-View-Controller (MVC).....	11
Repository Pattern.....	11
RESTful API Design.....	12
Input Validation.....	12
4. CONTEXT VIEWPOINT.....	13
4.1. Use-case Diagram.....	13
5. COMPOSITION(DECOMPOSITION) VIEWPOINT.....	14
5.1. System Architecture Diagram.....	14
a. Presentation Layer (React.js).....	14
b. Application Layer (Spring Boot).....	15
c. Data Layer (PostgreSQL).....	16
5.2. Module/ Component Decomposition.....	17

6. INFORMATION (DATA) VIEWPOINT.....	18
6.1. Entity Relationship Diagram(ERD).....	18
6.2. Data Dictionary.....	19
6.3. Database Design Notes.....	21
Database Management System.....	21
Database Normalization.....	21
Indexing Strategy.....	22
Data Integrity Rules.....	22
7. STRUCTURAL (CLASS) VIEWPOINT.....	23
8. INTERFACE VIEWPOINT.....	23
8.1 User Interface Design.....	23
8.2 Application Programming Interfaces (APIs).....	28
9. INTERACTION AND DYNAMIC VIEWPOINT.....	31
9.1 Sequence Diagram.....	31
9.2 Activity Diagram.....	32
9.3 State Chart Diagram.....	33
10. ALGORITHM (PROCESSING) VIEWPOINT.....	33
10.1 User Authentication Algorithm.....	34
Purpose: To verify user credentials and allow access to the system securely using JWT.....	34
Input:.....	34
Output:.....	34
Algorithm Steps:.....	34
Pseudocode:.....	35
10.2 Idea Submission Algorithm.....	35
Purpose: To allow students to submit innovative ideas linked to a problem.....	35
Input:.....	35
Output:.....	35
Steps:.....	35
Pseudocode:.....	36
10.3 Voting Algorithm.....	36
Purpose: To allow users to vote for ideas (like/dislike or upvote system).....	36
Input:.....	36
Output:.....	36
Steps:.....	36
Pseudocode:.....	36
10.4 Lecturer Feedback Algorithm.....	37
Purpose: To allow lecturers to provide guidance on student ideas.....	37
Input:.....	37
Output:.....	37
Steps:.....	37

Pseudocode:.....	37
11. Security and Deployment Viewpoint.....	38
11.1 Security Design.....	38
a. Authentication (JWT Security).....	38
b. Authorization (Role-Based Access Control).....	38
c. Password Encryption.....	38
d. Input Validation.....	38
e. Protection Against Attacks.....	39
f. Session Security.....	39
11.2 Deployment Viewpoint.....	39
11.2.1 Deployment Architecture.....	39
11.2.2 Deployment Environment.....	39
11.2.3 Deployment Strategy.....	39
11.2.4 Advantages of Deployment Design.....	39
12. Requirements-to-Design Traceability.....	40
12.1 Functional Requirements Traceability.....	40
12.2 Non-Functional Requirements Traceability.....	41
Declaration of Authorship and AI Use.....	42
A. Permitted vs. Not Permitted.....	42
B. AI Tools Used in This Submission.....	42
C. Student Attestation.....	42
Appendix - Primary Evidence (Required).....	43
E.1 Project Repository (Version Control).....	43
E.2 Design-to-Code Evidence.....	44

1. INTRODUCTION

1.1 Purpose

The purpose of Software Design Description(SDD) is to describe the architecture and design of the University Innovation Connect (UIC) proposed system. This SDD document clearly explains how the Software Requirement Specification(SRS) will be implemented through software components, user interfaces, database structures and system interactions.

This document serves as a technical guide for the development team and supervisor by providing a clear description of the system design and implementation approach. It ensures that the development process follows a structured design that satisfies both the functional and non-functional requirements of the system.

1.2 Scope

University Innovation Connect (UIC) is a web-based collaborative platform developed to improve innovation activities at the State University of Zanzibar (SUZA). The system provides a centralized environment where students can share innovative ideas, discuss academic or real-world challenges, and receive constructive feedback from lecturers.

The system supports user registration and authentication, idea submission, problem posting, commenting, voting, and lecturer feedback. It is designed to encourage collaboration between students and lecturers while reducing the loss of innovative ideas that are commonly shared through informal communication channels.

The scope of this design is limited to the web-based prototype implementation using React.js for the frontend, Spring Boot for the backend, and PostgreSQL as the database management system. Mobile applications, artificial intelligence features, advanced analytics, and third-party integrations are outside the scope of this project.

1.3 Definitions, Acronyms and Abbreviations

Term/ Acronym	Definition
UIC	University Innovation Connect
SUZA	State University of Zanzibar
SDD	Software Design Document
SRS	Software Requirements Specification
UI	User Interface
API	Application Programming Interface
REST	Representational State Transfer
HTTP	Hypertext Transfer Protocol
JSON	JavaScript Object Notation
DBMS	Database Management System
MVC	Model-View-Controller
ERD	Entity Relationship Diagram
CRUD	Create, Read, Update, Delete

1.4 References

The following documents and technologies were used in preparing the design of the University Innovation Connect (UIC) system:

- a. Software Requirements Specification (SRS) for *University Innovation Connect (UIC)*, Department of Computer Science and Information Technology, State University of Zanzibar, 2026.
- b. Project Proposal Document for *University Innovation Connect (UIC)*, Department of Computer Science and Information Technology, State University of Zanzibar, 2026.
- c. IEEE Std 1016-2009, *IEEE Standard for Information Technology-Systems Design-Software Design Descriptions*.

2. DESIGN STAKEHOLDERS AND THEIR CONCERNS

The design of the University Innovation Connect (UIC) system considers the needs and expectations of all stakeholders who will interact with or be affected by the system.

Understanding these concerns helps ensure that the system is functional, secure, maintainable, and easy to use.

STAKEHOLDER	KEY DESIGN CONCERNS
Students	Students require a simple, user-friendly, and responsive interface for registering, logging in, posting innovative ideas, viewing challenges, commenting, voting on ideas, and receiving feedback from lecturers.
Lecturers	Lecturers need an efficient platform for reviewing students' ideas, providing feedback, monitoring innovation activities, and guiding students throughout the innovation process. They also require secure access to their accounts and academic information.
System Administrator	The administrator is responsible for managing user accounts, monitoring system activities, maintaining data integrity, and ensuring the overall security and availability of the system.
State University of Zanzibar (SUZA)	The university expects a reliable and secure platform that promotes student innovation, improves collaboration between students and lecturers, supports academic activities, and enhances the management of innovative ideas within the institution.

3. DESIGN OVERVIEW AND RATIONALE

This chapter describes the overall software architecture of the University Innovation Connect (UIC) system and explains the design principles and patterns adopted during system development. The selected architecture aims to produce a system that is modular, secure, maintainable, scalable, and easy to understand.

3.1. Architectural Style

The University Innovation Connect (UIC) system adopts a **three-tier client-server architecture**. This architecture separates the system into three independent layers, each with a specific responsibility.

Presentation Layer (Frontend)

The presentation layer is developed using **React.js**. It provides the graphical user interface (GUI) through which students, lecturers, and administrators interact with the system. The frontend communicates with the backend through RESTful APIs using HTTP requests and responses in JSON format.

Application Layer (Backend)

The application layer is developed using **Spring Boot**. It contains the business logic responsible for processing user requests, validating data, enforcing system rules, authenticating users, and communicating with the database.

Data Layer (Database)

The data layer uses **PostgreSQL** as the relational database management system. It stores all application data, including user accounts, innovation ideas, comments, votes, feedback, and notifications.

Architecture Justification

The three-tier architecture was selected because it:

- Separates user interface, business logic, and data management.
- Improves system maintainability and readability.
- Simplifies testing and debugging.
- Allows independent modification of each layer without affecting the others.
- Supports future expansion of the system, such as adding a mobile application or additional services.

System Architecture

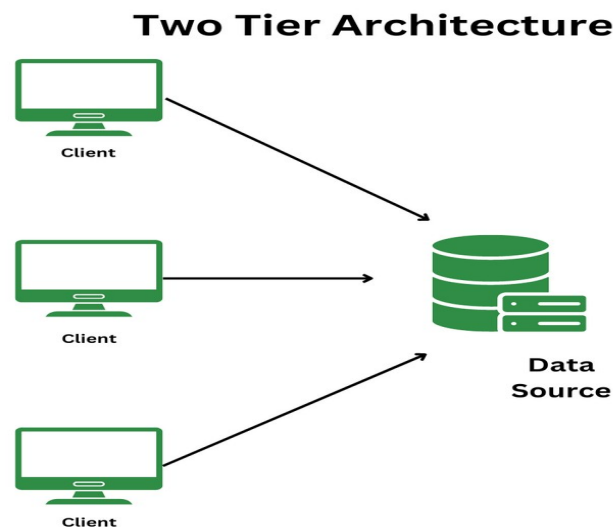


Figure 1: System Architecture

3.2 Design Principles and Patterns

Separation of Concerns (SoC)

Each layer of the system performs a specific responsibility. The frontend manages presentation, the backend handles business logic, and the database stores application data.

Single Responsibility Principle (SRP)

Each class performs one specific function. For example:

- Controllers handle HTTP requests.
- Services implement business logic.
- Repositories access database records.
- Entities represent database tables.

This makes the system easier to maintain and extend.

Don't Repeat Yourself (DRY)

Reusable methods and components are created whenever possible to reduce duplicate code and improve maintainability.

Model-View-Controller (MVC)

Spring Boot follows the MVC architectural pattern:

- **Model:** Entity classes and repositories.
- **View:** React frontend.
- **Controller:** REST Controllers that receive client requests.

Repository Pattern

Spring Data JPA repositories are used to separate database operations from business logic. This simplifies data access and improves maintainability.

RESTful API Design

Communication between the frontend and backend follows REST principles using standard HTTP methods:

- GET - Retrieve data
- POST - Create data
- PUT - Update data
- DELETE - Remove data

Input Validation

Both frontend and backend validate user inputs to prevent invalid data from entering the system.

4. CONTEXT VIEWPOINT

The Context Viewpoint describes the University Innovation Connect (UIC) system as a whole and illustrates how it interacts with external users. It identifies the actors that communicate with the system and the information exchanged between them.

The UIC system serves as a centralized platform that enables students and lecturers to collaborate on innovative ideas while administrators manage system operations.

4.1. Use-case Diagram

4.2. Describe External Entity and data exchange with the system

External Entity	Data Sent to the System	Data Received from the System
Students	Registration details, login credentials, innovation ideas, comments, votes, profile updates	Authentication results, innovation ideas, lecturer feedback, comments, notifications
Lecturers	Login credentials, comments, reviews, feedback, profile updates	Student ideas, innovation challenges, notifications, system messages
Administrator	User management requests, account updates, system configuration	User information, reports, system activity, management confirmation

5. COMPOSITION(DECOMPOSITION) VIEWPOINT

5.1. System Architecture Diagram

The University Innovation Connect (UIC) system follows a **three-tier client-server architecture** consisting of the Presentation Layer, Application Layer, and Data Layer.

The architecture separates the user interface, business logic, and data storage responsibilities. This improves system organization, maintainability, security, and future scalability.

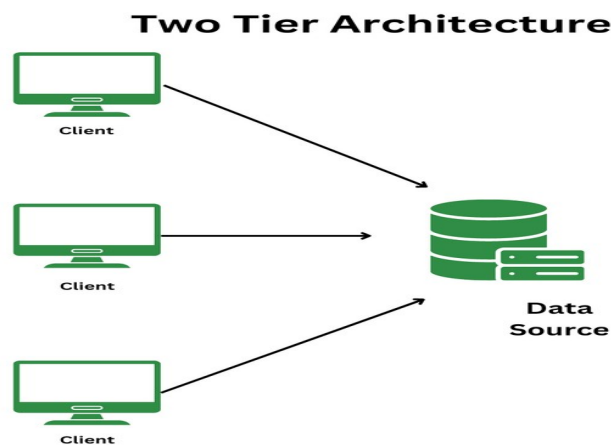


Figure 1: System Architecture

a. Presentation Layer (React.js)

The presentation layer provides the interface where users interact with the system.

Responsibilities include:

- Displaying web pages and forms
- Collecting user inputs
- Sending requests to the backend API
- Displaying system responses
- Managing user navigation

Main interfaces include:

- Registration page
- Login page
- Student dashboard
- Lecturer dashboard
- Idea submission page
- Comment and voting interface

b. Application Layer (Spring Boot)

The application layer contains the main business logic of the system.

Responsibilities include:

- Processing user requests
- Validating user information
- Managing authentication
- Handling idea submissions
- Processing comments and votes
- Managing lecturer feedback
- Communicating with the database

c. Data Layer (PostgreSQL)

The database layer stores and manages system information.

Main stored data includes:

- accounts
- Student problems
- Submitted ideas
- Comments
- Votes
- Lecturer feedback

5.2. Module/ Component Decomposition

The UIC system is divided into several modules. Each module has a specific responsibility and implements one or more functional requirements from the SRS.

Module / Component	Responsibility	Realises Requirement
Authentication Module	Handles user registration, login, password security, and user verification	FR-01 User Registration, FR-02 Login
User Management Module	Manages student, lecturer, and administrator profiles	FR-03 User Profile Management
Problem Management Module	Allows students to post academic or real-life challenges	FR-04 Problem Posting
Idea Management Module	Enables students to submit innovative solutions and manage ideas	FR-05 Idea Submission
Idea Viewing Module	Allows users to view submitted ideas and challenges	FR-06 View Ideas
Comment Management Module	Allows users to discuss ideas through comments	FR-07 Comment System
Voting Module	Allows users to like or vote for useful ideas	FR-08 Idea Voting
Feedback Module	Allows lecturers to provide guidance and evaluation	FR-09 Lecturer Feedback
Notification Module	Provides users with updates about activities	FR-10 Notifications
Administration Module	Allows administrators to manage users and system data	FR-11 User Management

6. INFORMATION (DATA) VIEWPOINT

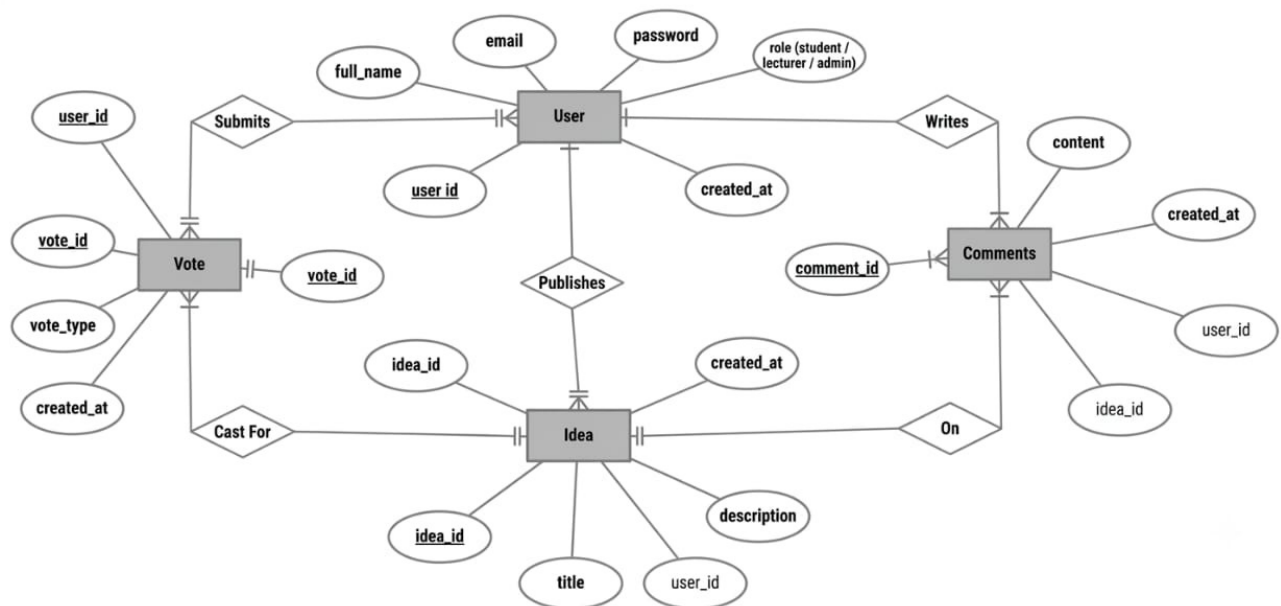
The Information Viewpoint describes how data is organized, stored, and managed in the University Innovation Connect (UIC) system.

The system uses a relational database design implemented using PostgreSQL.

The database stores information about:

- Users
- Problems
- Ideas
- Comments
- Votes
- Lecturer feedback

6.1. Entity Relationship Diagram(ERD)



6.2. Data Dictionary

a. users

column_name	data_type	is_nullable	column_default
id	bigint	NO	
created_by	character varying	YES	
created_date	timestamp without time zone	YES	
last_modified_by	character varying	YES	
last_modified_date	timestamp without time zone	YES	
email	character varying	YES	
first_name	character varying	YES	
last_name	character varying	YES	
password	character varying	YES	
role	character varying	YES	
(10 rows)			

b. challenge

column_name	data_type	is_nullable	column_default
id	bigint	NO	
created_by	character varying	YES	
created_date	timestamp without time zone	YES	
last_modified_by	character varying	YES	
last_modified_date	timestamp without time zone	YES	
description	character varying	YES	
sector	character varying	YES	
title	character varying	YES	
user_id	bigint	YES	
(9 rows)			

c. comments

column_name	data_type	is_nullable	column_default
id	bigint	NO	
created_by	character varying	YES	
created_date	timestamp without time zone	YES	
last_modified_by	character varying	YES	
last_modified_date	timestamp without time zone	YES	
message	character varying	YES	
idea_id	bigint	YES	
user_id	bigint	YES	
(8 rows)			

d. ideas

```
WHERE table_name = 'ideas';
```

column_name	data_type	is_nullable	column_default
id	bigint	NO	
created_by	character varying	YES	
created_date	timestamp without time zone	YES	
last_modified_by	character varying	YES	
last_modified_date	timestamp without time zone	YES	
category	character varying	YES	
description	character varying	YES	
status	character varying	YES	
title	character varying	YES	
user_id	bigint	YES	

(10 rows)

e. votes

```
WHERE table_name = 'votes';
```

column_name	data_type	is_nullable	column_default
id	bigint	NO	
created_by	character varying	YES	
created_date	timestamp without time zone	YES	
last_modified_by	character varying	YES	
last_modified_date	timestamp without time zone	YES	
idea_id	bigint	YES	
user_id	bigint	YES	

(7 rows)

f. feedbacks

column_name	data_type	is_nullable	column_default
id	bigint	NO	
created_by	character varying	YES	
created_date	timestamp without time zone	YES	
last_modified_by	character varying	YES	
last_modified_date	timestamp without time zone	YES	
comment	character varying	YES	
idea_id	bigint	YES	
lecturer_id	bigint	YES	

(8 rows)

6.3. Database Design Notes

Database Management System

The system uses: **PostgreSQL** because it provides:

- Reliability
- Security
- Support for relational relationships
- Good performance
- Compatibility with Spring Boot

Database Normalization

The database follows **Third Normal Form (3NF)**.

Normalization ensures:

- Reduced data duplication
- Improved data consistency
- Easier database maintenance

Indexing Strategy

Indexes will be created on frequently searched fields:

- User email
- Idea title
- Problem title

This improves search performance.

Data Integrity Rules

The database applies:

- Primary key constraints
- Foreign key relationships
- Unique email validation
- NOT NULL restrictions
- Referential integrity rules

7. STRUCTURAL (CLASS) VIEWPOINT

8. INTERFACE VIEWPOINT

8.1 User Interface Design

The UIC system provides different interfaces depending on the user's role.

The main user interfaces include:

1. **Registration Interface:** Allows new students to create accounts.

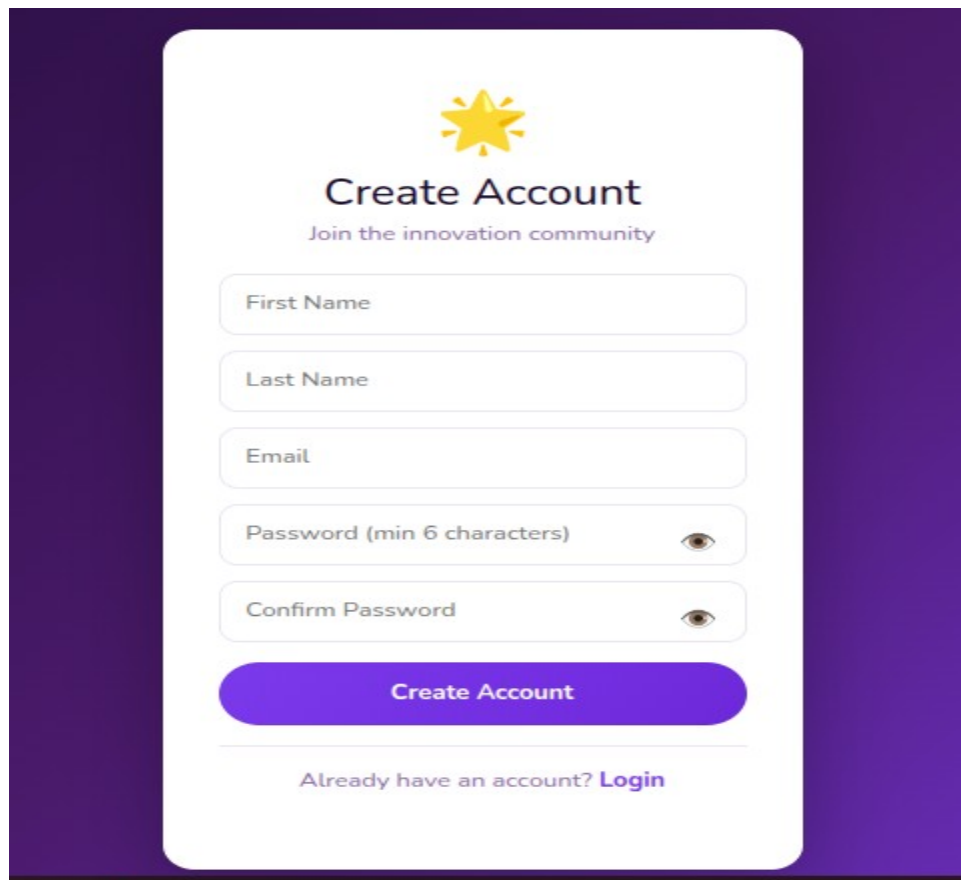
The image shows a registration form titled "Create Account" with the subtitle "Join the innovation community". At the top is a yellow star icon. The form contains five input fields: "First Name", "Last Name", "Email", "Password (min 6 characters)", and "Confirm Password". Each password field has an eye icon for toggling visibility. Below the fields is a large purple "Create Account" button. At the bottom, there is a link that says "Already have an account? Login". The entire form is centered on a dark purple background.

Figure 3:Registration page

2. **Login Interface:** Allows registered users to access the system.

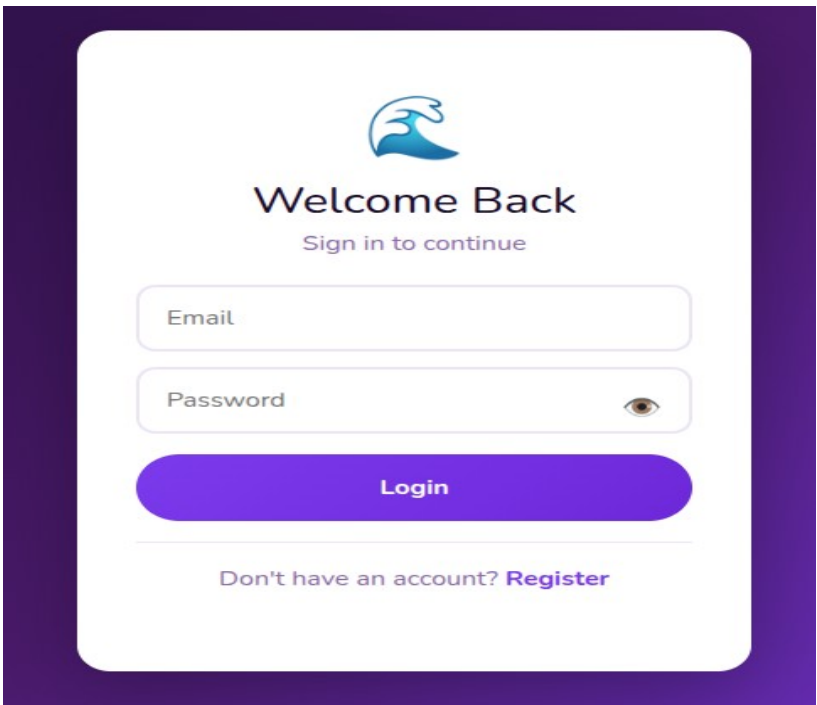


Figure4:Login page

3. **Student Dashboard:** The student dashboard provides access to innovation activities. Features like View problems, submit ideas, View ideas, Comment on ideas, Receive lecturer feedback, and Vote for ideas

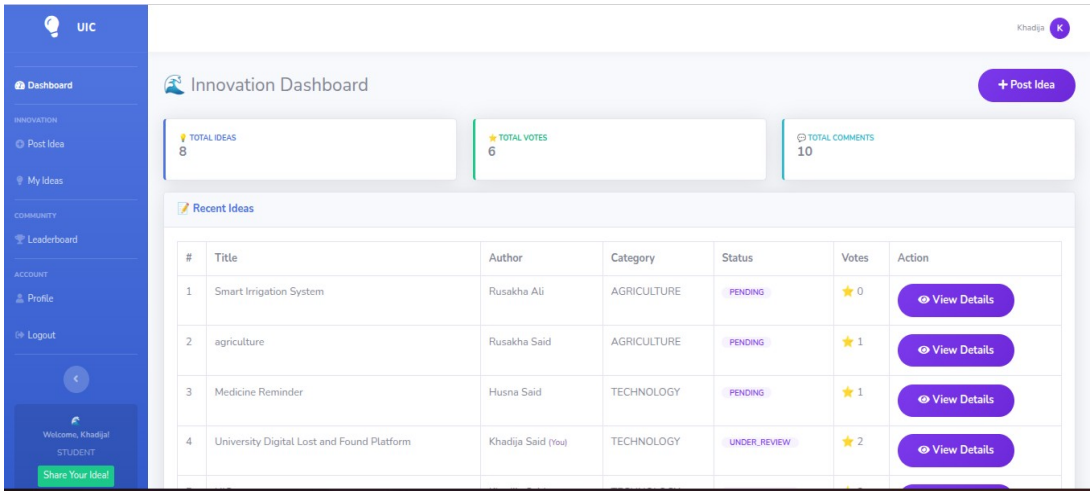


Figure 5:Student Dashboard

4. **Lecturer Dashboard:** The lecturer dashboard supports academic guidance. Features like View student ideas, Review innovation proposals, Add feedback, Monitor discussions

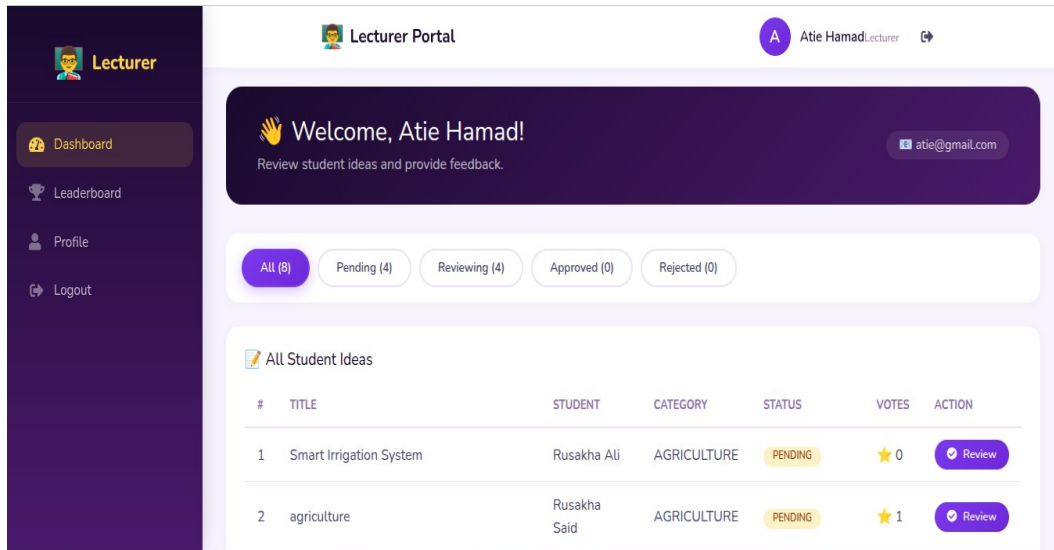


Figure6:Lecture Dashboard

5. Idea Submission Interface

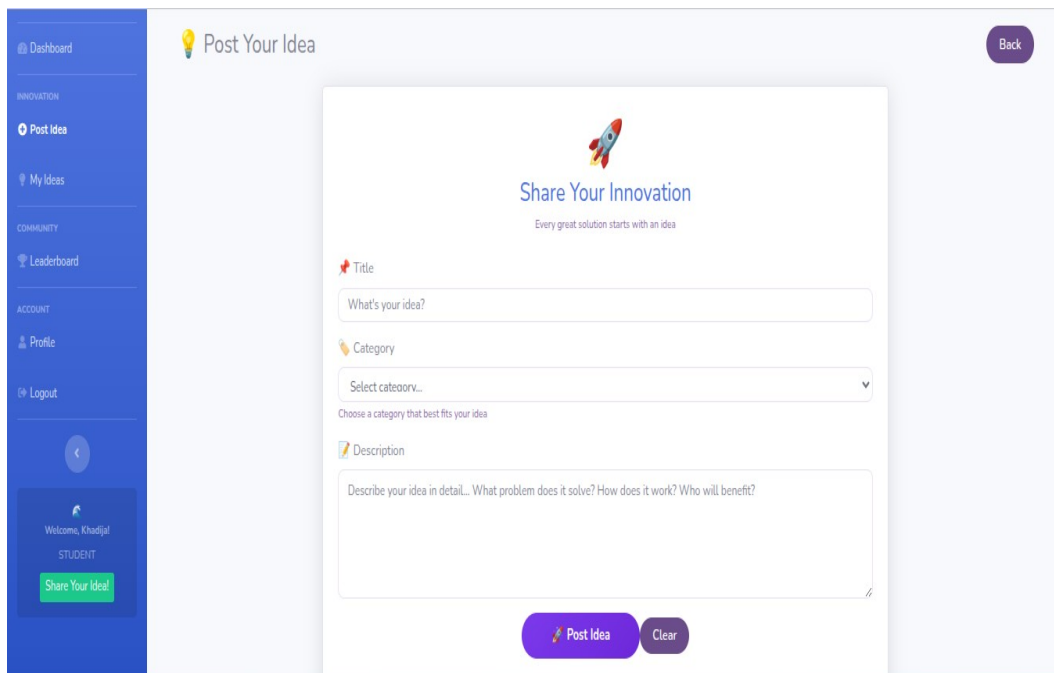


Figure7:Post Idea page

6. Comment Interface and Voting Interface

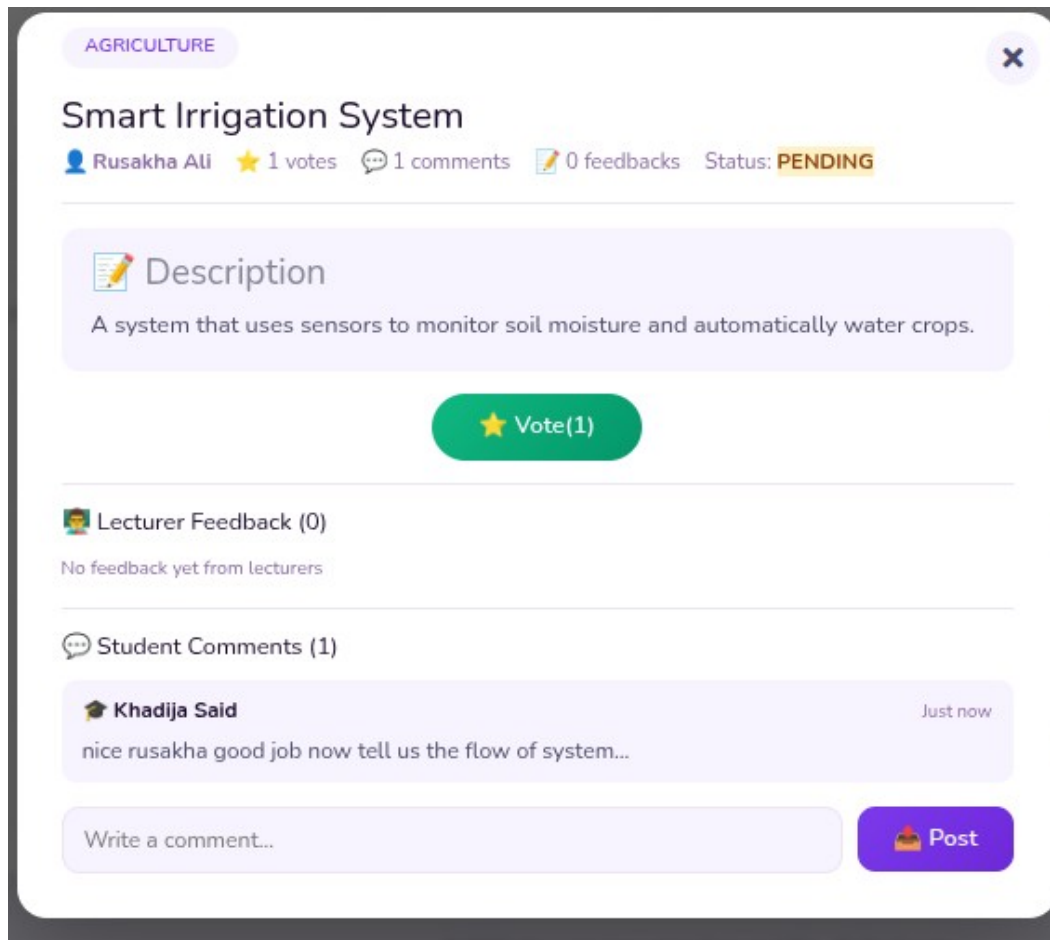


Figure8:Comments and Votes interface

8.2 Application Programming Interfaces (APIs)

The frontend communicates with the Spring Boot backend through RESTful APIs.

All communication uses:

- HTTP methods
- JSON data format
- JWT authentication

API Endpoints:

Authentication APIs

Endpoint	Method	Request/Response	Purpose
/api/v1/auth/register	POST	User JSON data	Create new account
/api/v1/auth/login	POST	Email and password	Authenticate user

User APIs

Endpoint	Method	Request/Response	Purpose
/api/v1/users	GET	User list	Retrieve users
/api/v1/users/{id}	GET	User ID	View profile
/api/v1/users/{id}	PUT	User data	Update profile

Problem APIs

Endpoint	Method	Request/Response	Purpose
/api/v1/problems	GET	Problem list	View problems
/api/v1/problems	POST	Problem JSON	Create problem
/api/v1/problems/{id}	PUT	Problem data	Update problem
/api/v1/problems/{id}	DELETE	Problem ID	Delete problem

Idea APIs

Endpoint	Method	Request/Response	Purpose
/api/v1/ideas	GET	Idea list	View ideas
/api/v1/ideas	POST	Idea JSON	Submit idea
/api/v1/ideas/{id}	GET	Idea ID	View idea details

Comment APIs

Endpoint	Method	Request/Response	Purpose
/api/v1/comments	POST	Comment JSON	Add comment
/api/v1/comments/{ideaId}	GET	Idea ID	View comments

Voting APIs

Endpoint	Method	Request/Response	Purpose
/api/v1/votes	POST	Vote data	Vote for idea
/api/v1/votes/{ideaId}	GET	Idea ID	View votes

Feedback APIs

Endpoint	Method	Request/Response	Purpose
/api/feedback	POST	Feedback JSON	Lecturer feedback
/api/feedback/{ideaId}	GET	Idea ID	View feedback

9. INTERACTION AND DYNAMIC VIEWPOINT

9.1 Sequence Diagram

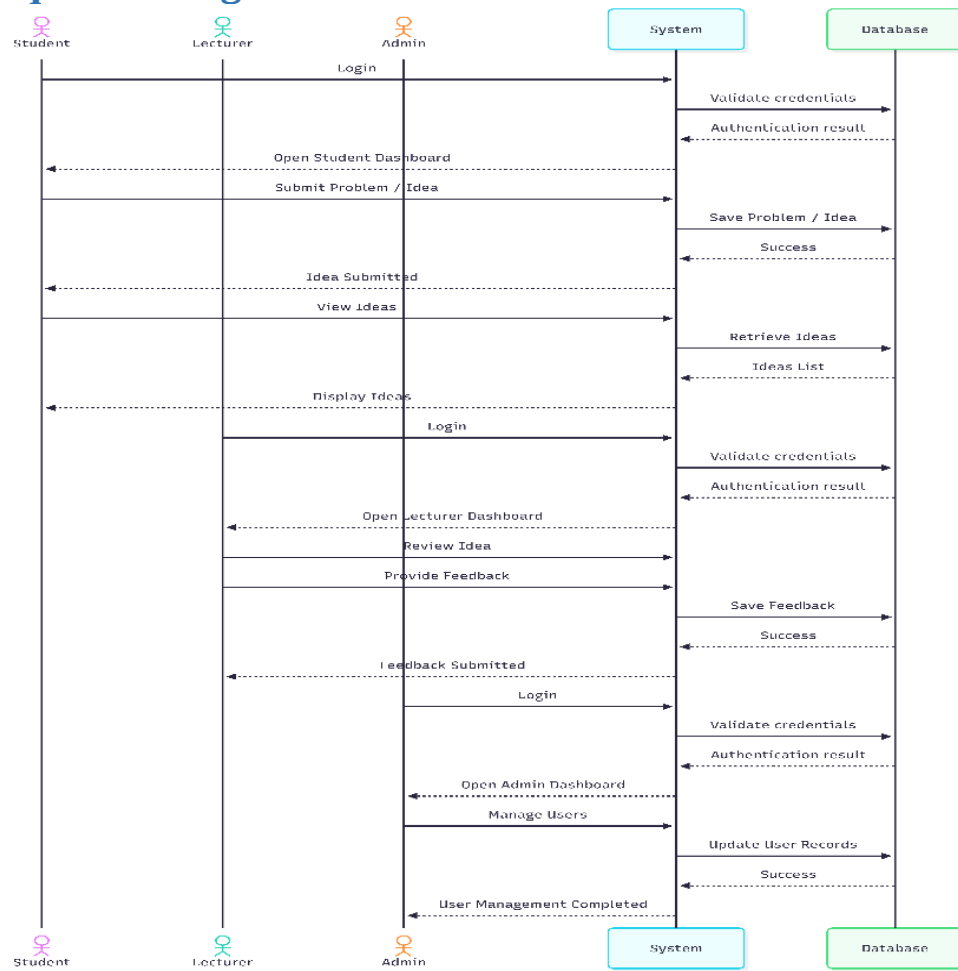


Figure9:Sequence diagram

9.2 Activity Diagram

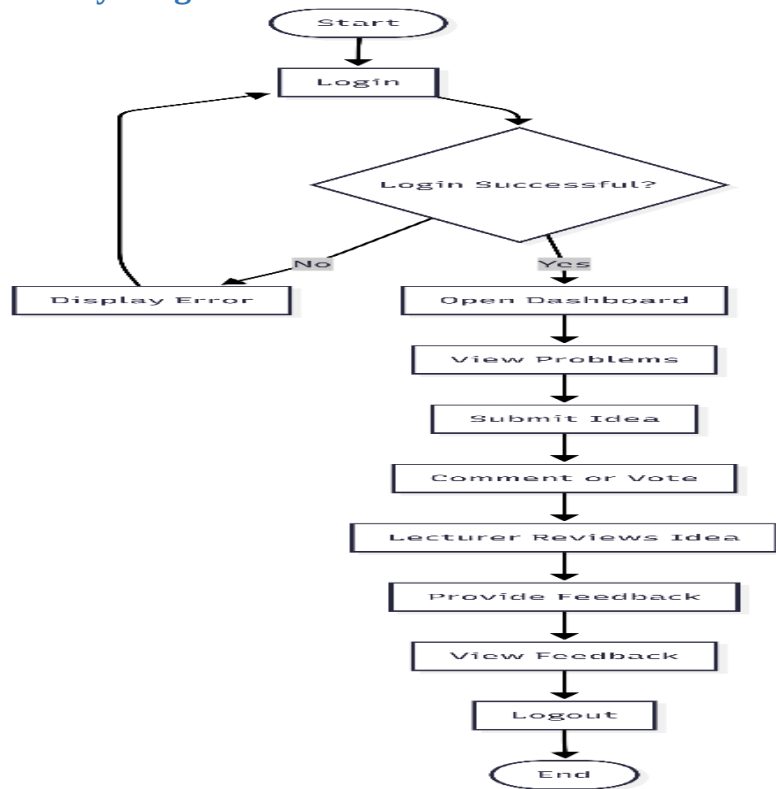


Figure10:Sequence Diagram

9.3 State Chart Diagram

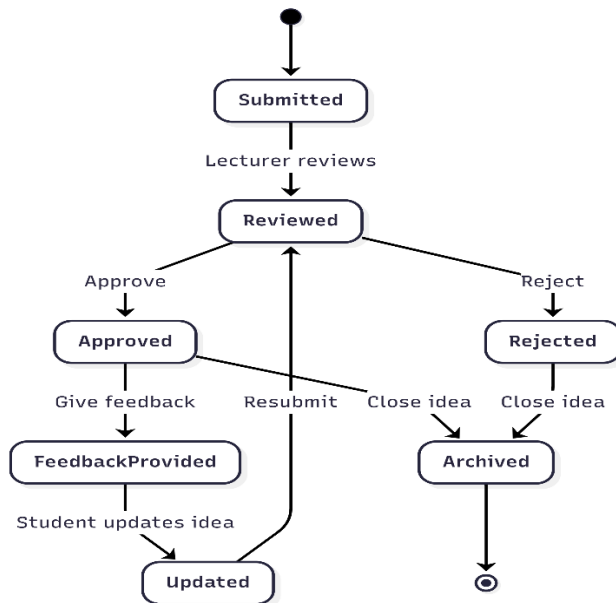


Figure11:State Chart Diagram

10. ALGORITHM (PROCESSING) VIEWPOINT

The Algorithm (Processing) Viewpoint describes the detailed logic and step-by-step procedures used in the University Innovation Connect (UIC) system to perform key operations. These algorithms explain how inputs are processed into outputs through system rules implemented in the backend (Spring Boot).

The main system algorithms include:

- User Authentication Algorithm
- Idea Submission Algorithm
- Voting Algorithm
- Lecturer Feedback Algorithm

10.1 User Authentication Algorithm

Purpose: To verify user credentials and allow access to the system securely using JWT.

Input:

- Email
- Password

Output:

- JWT Token (if successful)
- Error message (if failed)

Algorithm Steps:

1. User enters email and password.
2. System sends request to backend.
3. Backend searches user by email in database.
4. If user does not exist → return "User not found".
5. If password does not match → return "Invalid credentials".
6. If valid:
 - o Generate JWT token
 - o Assign user role (Student/Lecturer/Admin)
7. Return token to frontend.
8. User is redirected to dashboard.

Pseudocode:

```
IF user EXISTS in database THEN
  IF password is correct THEN
    GENERATE JWT token
    RETURN success response + token
  ELSE
    RETURN "Invalid password"
  ENDIF
ELSE
  RETURN "User not found"
ENDIF
```

10.2 Idea Submission Algorithm

Purpose: To allow students to submit innovative ideas linked to a problem.

Input:

- Idea title
- Idea description
- Problem ID
- User ID

Output:

- Saved idea confirmation

Steps:

1. Student fills idea form.
2. System validates input fields.
3. Check if user is authenticated.
4. Verify related problem exists.
5. Save idea in database.
6. Link idea to user and problem.
7. Return success message.

Pseudocode:

```
IF user IS authenticated THEN
  IF problem EXISTS THEN
    CREATE new Idea
    ASSIGN idea to user
    ASSIGN idea to problem
    SAVE idea in database
    RETURN success message
  ELSE
    RETURN "Problem not found"
  ENDIF
ELSE
  RETURN "Unauthorized access"
ENDIF
```

10.3 Voting Algorithm

Purpose: To allow users to vote for ideas (like/dislike or upvote system).

Input:

- User ID
- Idea ID

Output:

- Vote confirmation

Steps:

1. User selects idea.
2. System checks authentication.
3. Check if user already voted.
4. If not voted:
 - o Create vote record
 - o Save to database
5. If already voted:
 - o Remove or ignore duplicate vote
6. Update vote count.

Pseudocode:

```
IF user IS authenticated THEN
  IF user HAS NOT voted on idea THEN
    CREATE vote
    SAVE vote
    RETURN "Vote recorded"
  ELSE
    RETURN "Already voted"
  ENDIF
ELSE
  RETURN "Login required"
ENDIF
```

10.4 Lecturer Feedback Algorithm

Purpose: To allow lecturers to provide guidance on student ideas.

Input:

- Lecturer ID
- Idea ID
- Feedback message

Output:

- Feedback confirmation

Steps:

1. Lecturer logs in.
2. Lecturer selects idea.
3. System validates lecturer role.
4. Save feedback linked to idea.
5. Notify student.
6. Display feedback on idea page.

Pseudocode:

```
IF user.role == "LECTURER" THEN
  IF idea EXISTS THEN
    CREATE feedback
    LINK feedback to idea
    SAVE feedback
    RETURN "Feedback submitted"
  ELSE
    RETURN "Idea not found"
  ENDIF
ELSE
  RETURN "Access denied"
ENDIF
```

11. Security and Deployment Viewpoint

11.1 Security Design

The UIC system implements several security mechanisms to protect user data and ensure safe system operation.

a. Authentication (JWT Security)

- Users must log in using email and password.
- Prevents unauthorized access.

b. Authorization (Role-Based Access Control)

The system defines three roles and Access is restricted based on role.:

Role	Permissions
Student	Submit ideas, vote, comment
Lecturer	View ideas, give feedback
Admin	Manage users and system

c. Password Encryption

- Passwords are encrypted using **BCrypt hashing**.
- Plain text passwords are never stored in the database.

d. Input Validation

- All user inputs are validated in frontend and backend.
- Prevents invalid or malicious data entry.

e. Protection Against Attacks

The system protects against:

- SQL Injection (using Spring Data JPA)
- Cross-Site Scripting (XSS)
- Cross-Site Request Forgery (CSRF via tokens)

f. Session Security

- JWT tokens expire after a defined time.
- Users must re-login after expiration.

11.2 Deployment Viewpoint

The deployment architecture describes how the UIC system is hosted and accessed.

11.2.1 Deployment Architecture

The system follows a **3-tier deployment model**:

11.2.2 Deployment Environment

Component	Technology
Frontend	React.js
Backend	Spring Boot
Database	PostgreSQL
Hosting	Local server / university server
API Communication	REST + JSON

11.2.3 Deployment Strategy

- Frontend deployed on web server (e.g., Nginx or Vercel)
- Backend deployed on Spring Boot server (Tomcat)
- Database hosted on PostgreSQL server
- Communication through secure HTTPS

11.2.4 Advantages of Deployment Design

- Easy scalability
- Separation of concerns
- Independent deployment of frontend and backend
- Secure data management
- Suitable for academic prototype

12. Requirements-to-Design Traceability

This chapter ensures that every functional and non-functional requirement defined in the Software Requirements Specification (SRS) for the **University Innovation Connect (UIC)** system is properly addressed in the Software Design Description (SDD).

It provides a clear mapping between:

- SRS Requirements (What the system should do)
- Design Components (How the system will be implemented)

This ensures **completeness, consistency, and traceability** of the system design.

12.1 Functional Requirements Traceability

SRS Requirement	Requirement Description	Design Element(s)	SDD Section
FR-01	User registration and login system	User class, Authentication algorithm, JWT security, UserController, AuthController	Chapter 7, 10, 11
FR-02	Students can post problems/challenges	Problem class, ProblemController, ProblemService	Chapter 7, 8, 10
FR-03	Students can submit innovative ideas	Idea class, IdeaController, IdeaService, Idea submission algorithm	Chapter 7, 8, 10
FR-04	Users can comment on ideas	Comment class, CommentController, CommentService	Chapter 7, 8
FR-05	Users can vote for ideas	Vote class, VoteController, VoteService, Voting algorithm	Chapter 7, 10
FR-06	Lecturers can provide feedback	Feedback class, FeedbackController, FeedbackService, Feedback algorithm	Chapter 7, 8, 10
FR-07	Users can view ideas and feedback	IdeaController (GET), FeedbackController (GET), React UI dashboards	Chapter 8
FR-08	System role management (Student/Lecturer/Admin)	User role attribute, Authorization module (JWT + RBAC)	Chapter 11
FR-09	System authentication security	JWT Authentication, AuthService	Chapter 10, 11
FR-10	Data persistence for all activities	PostgreSQL Database, Repository layer	Chapter 7, 11

12.2 Non-Functional Requirements Traceability

SRS Requirement	Requirement Description	Design Element(s)	SDD Section
NFR-01	Security of user data	JWT authentication, password encryption (BCrypt), role-based access control	Chapter 11
NFR-02	System usability	React UI dashboards (Student/Lecturer/Admin), simple interface design	Chapter 8
NFR-03	Performance efficiency	Spring Boot REST API, layered architecture, optimized database queries	Chapter 3, 7
NFR-04	Maintainability	MVC architecture, service-repository pattern, separation of concerns	Chapter 3, 7
NFR-05	Scalability	Three-tier architecture (Frontend, Backend, Database separation)	Chapter 3, 11
NFR-06	Reliability	Error handling, validation mechanisms, transaction management	Chapter 10, 11
NFR-07	Portability	Web-based system accessible via browser (React frontend)	Chapter 8, 11
NFR-08	Data integrity	PostgreSQL constraints, relational design, validation rules	Chapter 6, 11

Declaration of Authorship and AI Use

This declaration forms part of the Software Design Description (SDD) for the University Innovation Connect (UIC) project. It confirms that the design decisions, analysis, and implementation presented in this document are the original work of the project authors. Any use of Artificial Intelligence (AI) tools has been appropriately declared and was limited to acceptable academic purposes

A. Permitted vs. Not Permitted

Permitted (must still be declared)	Not Permitted
Explaining software engineering concepts and technologies	Generating the project's problem statement or objectives without student input
Assisting with programming syntax or boilerplate code that was reviewed and adapted by the students	Producing the system architecture or design decisions without student understanding
Helping debug code written by the students	Completing the entire project document without student contribution
Improving grammar, formatting, and clarity of student-written content	Producing analysis or design decisions that the students cannot explain during the viva

B. AI Tools Used in This Submission

AI Tool / Service	Specifically Used For	Section(s) Affected
ChatGPT (OpenAI)	Explaining software engineering concepts, improving grammar, refining technical writing, generating example UML diagram structures, and assisting with Spring Boot and React debugging	Chapters 1–12, diagrams, documentation, and code debugging

Note: All architectural decisions, database design, UML diagrams, software implementation, testing, and project analysis were reviewed, modified, and understood by the students before inclusion in this document.

C. Student Attestation

We confirm that the intellectual content of this Software Design Description (SDD), including the analysis, software design decisions, implementation strategy, and technical judgments, is our own work. Any assistance received from Artificial Intelligence (AI) tools has been fully declared in this document and was used only for learning, language improvement, debugging assistance, and formatting support.

We understand that we may be required to explain and defend any part of this submission during the oral examination (viva), and failure to do so may be treated as academic misconduct.

Student 1 (Author): Khadija Ali Said

Student 2 (Author): Rahma Suleiman Abdalla

Appendix - Primary Evidence (Required)

This appendix provides evidence of the development activities carried out during the implementation of the **University Innovation Connect (UIC)** project.

E.1 Project Repository (Version Control)

Field	Value
Repository URL	https://github.com/Rahma-suleiman/UniInivationConnect_Backend https://github.com/KhadijaSaid07/University-Innovation-Connect.git
Date of First Commit	June 2026
Approximate Number of Commits	33 commits
Supervisor Granted Access?	Yes

E.2 Design-to-Code Evidence

Design Element (from SDD)	Implemented In (File / Commit)	Realises (Requirement ID)
User Authentication Module (JWT Login)	src/main/java/com/uni/innovationConnect/auth/AuthController.java	FR-01
User Registration Module	AuthService.java	FR-01
User Entity	User.java	FR-01
Problem Management Module	ProblemController.java, ProblemService.java	FR-02
Idea Management Module	IdeaController.java, IdeaService.java	FR-03
Comment Management Module	CommentController.java, CommentService.java	FR-04
Voting Module	VoteController.java, VoteService.java	FR-05
Lecturer Feedback Module	FeedbackController.java, FeedbackService.java	FR-06
Role-Based Security (JWT & Spring Security)	SecurityConfig.java, JwtAuthenticationFilter.java	FR-08, NFR-01
PostgreSQL Database Design	Entity classes (User.java, Problem.java, Idea.java, Comment.java, Vote.java, Feedback.java)	